

자료구조

2분반

HW3

보고서

담당교수님 : 장형수 교수님

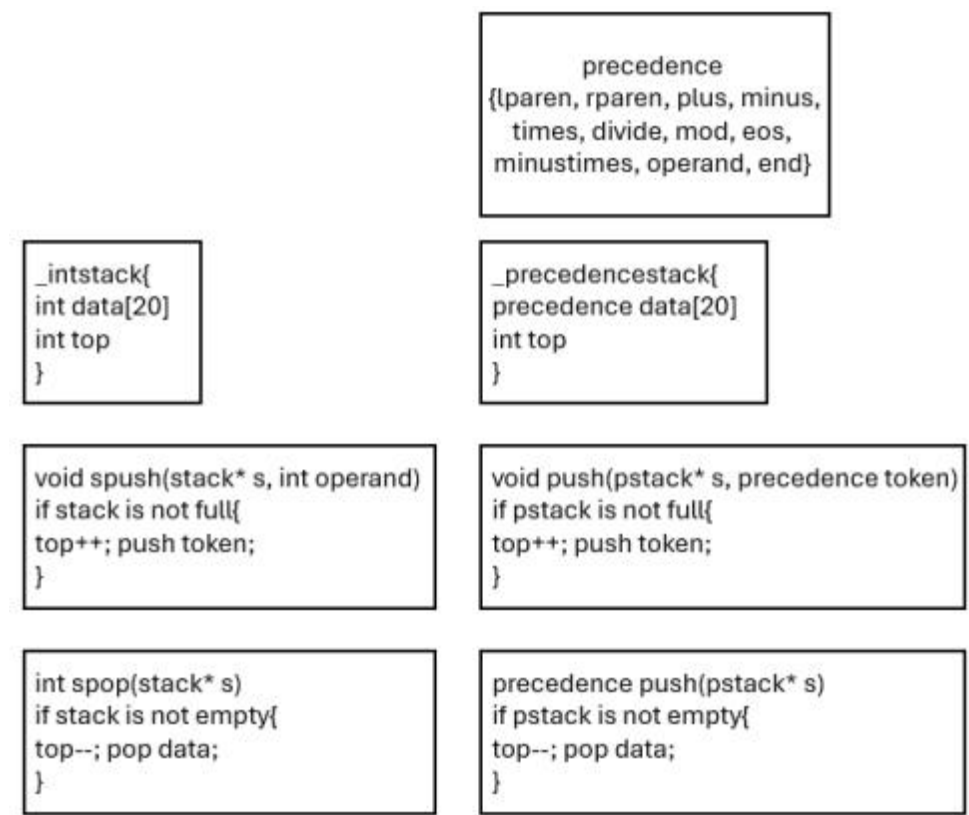
학과 : 컴퓨터공학과

학번: 20211562

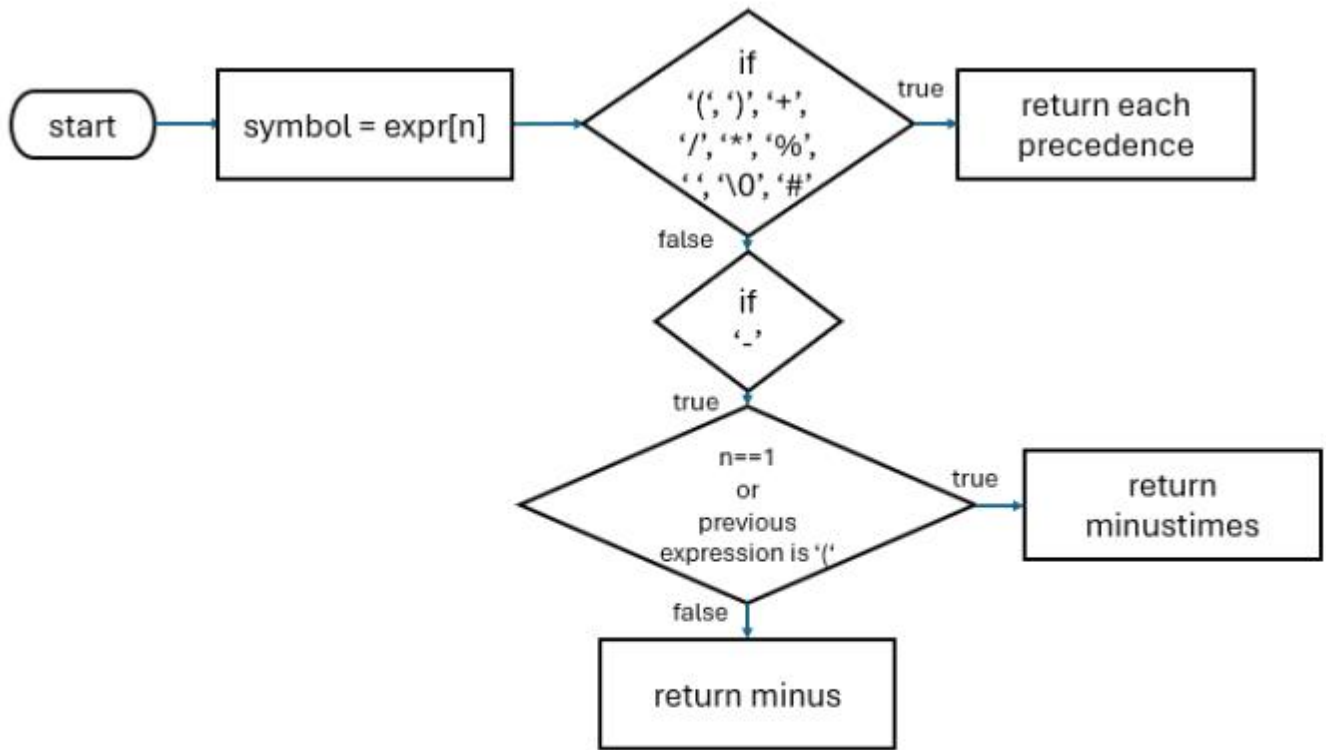
이름: 이동협

문제1

문제 1은 infix expression을 input으로 받았을 때 Postfix expression과 계산된 결과를 출력하는데, operator로 +,-,/,*,%만 입력받는 것이 아닌 integer 자체의 -(마이너스)값을 고려하여 postfix expression과 계산된 결과를 연산하면 되는 문제이다.

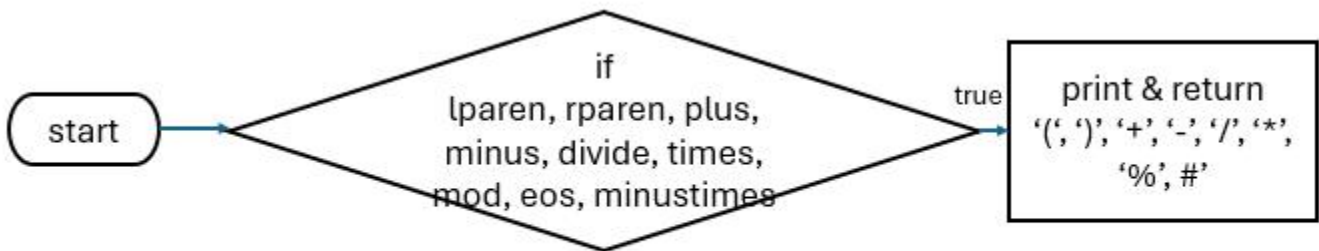


위의 code에서 동작하기 위해 필요한 ADT와 function을 구현한 것이다. 각 operator, operand, parenthesis의 precedence를 계산하기 위한 precedence 자료형, int stack, precedence stack, 그리고 이 두 stack의 push, pop 함수를 각각 구현하였다.



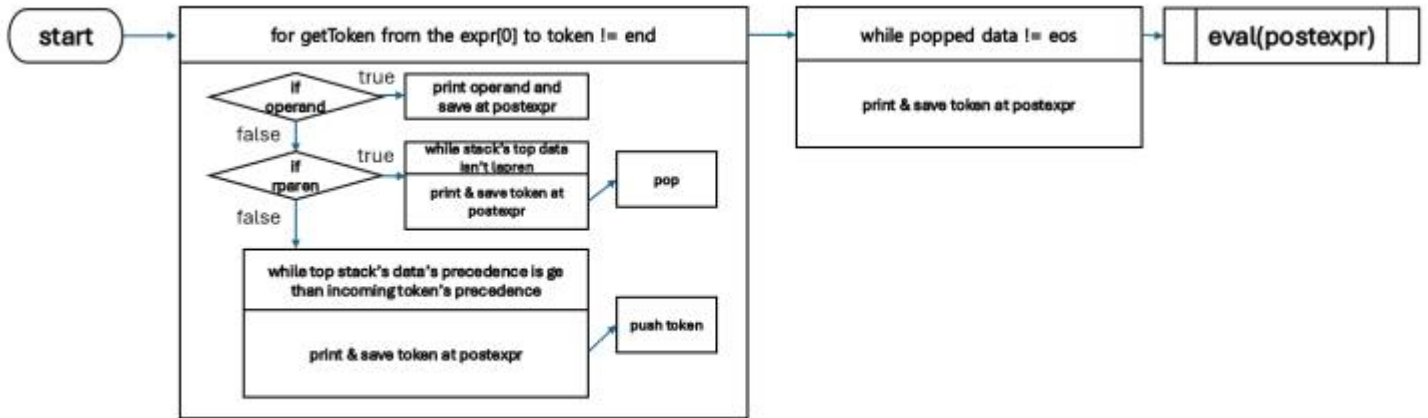
```
precedence getToken(char *symbol, int *n, char* expr)
```

getToken function은 symbol, n, expr을 인자로 입력받는다. symbol에 n번째 expr의 char을 저장하면서 n++시킨다. 이후 symbol이 '()+/*% \0#' 중 하나면 각각에 알맞는 precedence 값을 return한다. 이때 symbol이 -고 이것이 expr의 첫번째 char이거나 n-1번째 char이 (면 minustimes(빼기가 아닌 마이너스)를 return하고 아니라면 minus를 리턴한다.



```
char printToken(precedence token)
```

printToken function은 precedence 자료형인 token을 인자로 입력받아 각 token에 맞는 char값을 출력하면서 동시에 return한다.



```
void postfix(char* expr)
```

postfix function은 expression을 인자로 받는다. getToken으로 expression의 char을 하나씩 symbol에 저장하고 symbol에 맞는 precedence 값(token)으로 연산한다.

이후 1.2.3.을 expr끝까지 getToken할 때까지 반복한다.

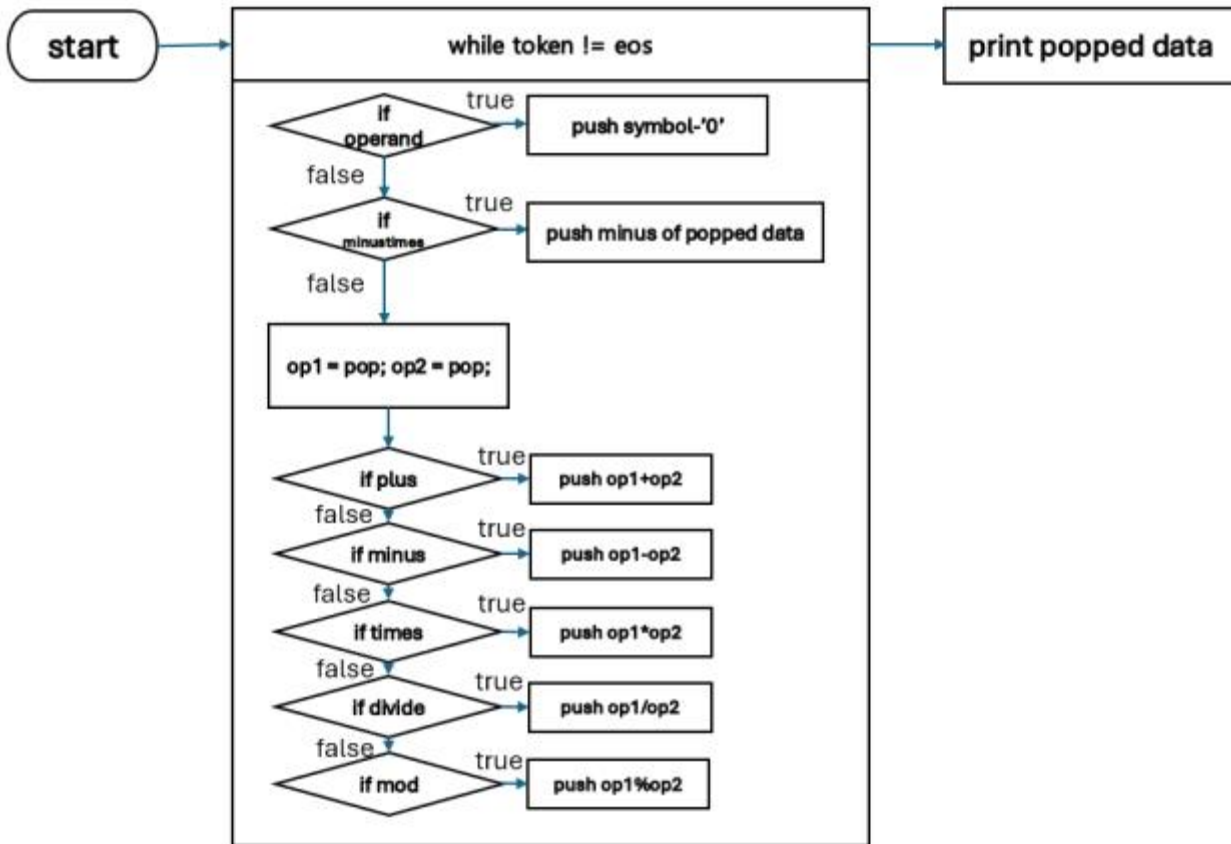
1.token이 operand인 경우 operand를 출력하고 postexpr(string)에 저장한다.

2.token이 rparen(')')인 경우 stack의 top data가 lparen('(')이 아닐 때까지 postexpr에 stack을 pop한 값을 저장한다. 이후 stack을 pop해서 lparen을 제거한다.

3.token이 위에 해당하지 않는 경우 stack top data의 precedence가 token의 precedence보다 크거나 같을 때까지 postexpr에 stack을 pop한 값을 저장한다. 이후 stack에 token을 push한다.

이후 stack에서 pop한 값이 eos가 아닐 때까지 stack에서 pop한 값을 postexpr에 저장한다.

이후 구한 postexpression을 계산하기 위해 eval()을 호출한다.



```
void eval(char* expr)
```

eval function은 post expression을 인자로 받아 그 결과를 계산하는 것이다.

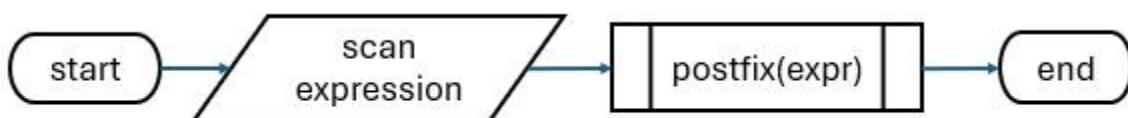
이도 동일하게 getToken으로 expression을 eos 전까지 하나하나 token으로 바꿔서

1. token이 operand면 stack에 symbol-'0' 값을 push한다.

2. token이 minustimes면 stack에서 pop한 값에 -를 곱하여 다시 push한다.

3. token이 위의 두 경우가 아니라면, op1에 stack에서 pop한 값을, op2에 stack에서 pop한 값을 저장한다. 이후 token이 plus면 stack에 op1+op2를 push, minus면 op1-op2를 push, times면 op1*op2를 push, divide면 op1/op2를 push, mod면 op1%op2를 push해준다.

while loop이 끝나면 stack에서 pop한 data를 출력한다.(이는 post expression의 연산 결과값이다.)



```
int main()
```

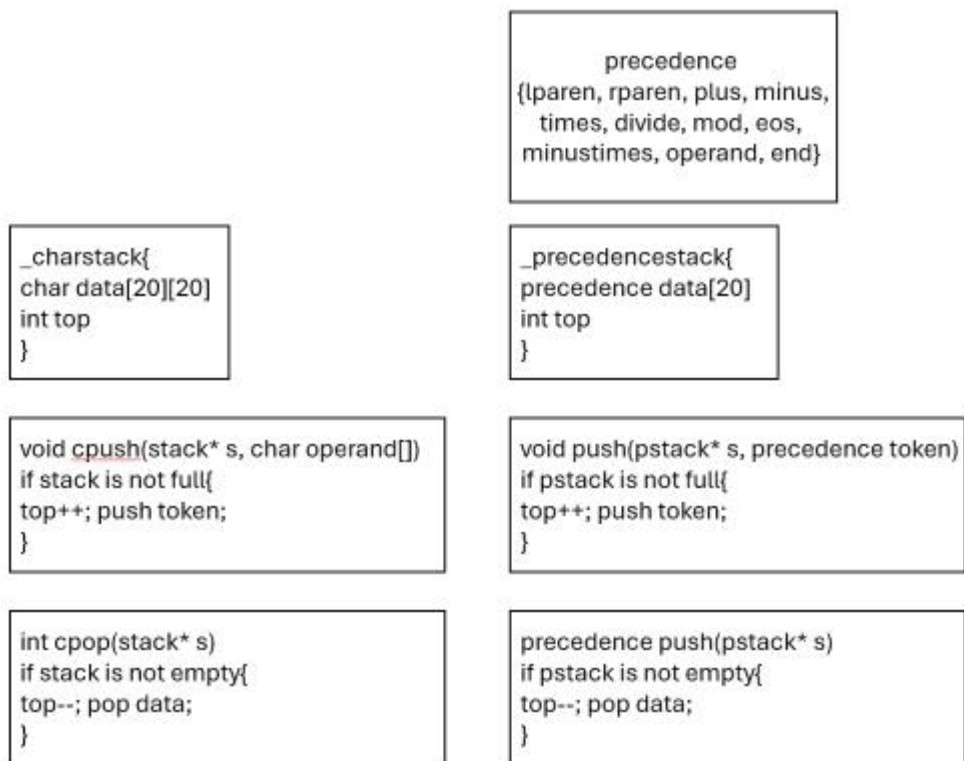
main function에서는 infix expression을 입력받아 저장하고 이를 인자로 postfix function을 호출한다.

```
cse20211562@cspro:~/structure/hw3$ ./a.out
Input: -6
Postfix: 6#
Result: -6
cse20211562@cspro:~/structure/hw3$ ./a.out
Input: (1-(-3))-5)
Postfix: 13#-5-
Result: -1
cse20211562@cspro:~/structure/hw3$ ./a.out
Input: 3*2+4*(5-1)
Postfix: 32*451-*+
Result: 22
```

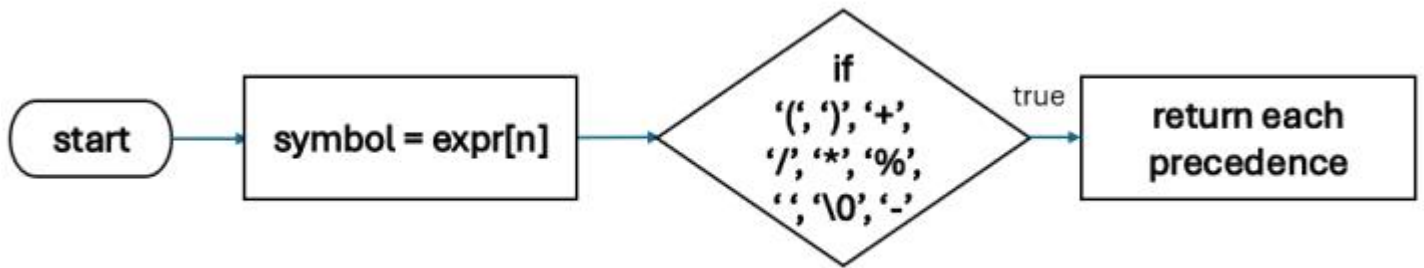
이는 제시된 출력 예시를 만족한다.

문제2

문제 2는 infix expression을 input으로 받아 prefix expression을 출력하는데, postfix expression을 구하는 방식을 사용하면 안 되는 문제이다.

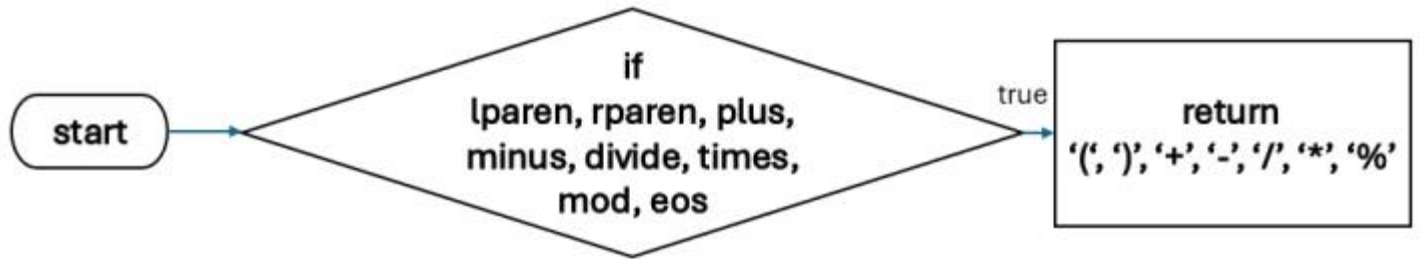


문제 2에서는 문제를 해결하기 위해 문제1과 동일한 precedence 자료형과 precedence stack, precedence stack의 push, pop function을 구현했고 string을 저장하는 charstack, charstack의 push, pop function 또한 구현했다.



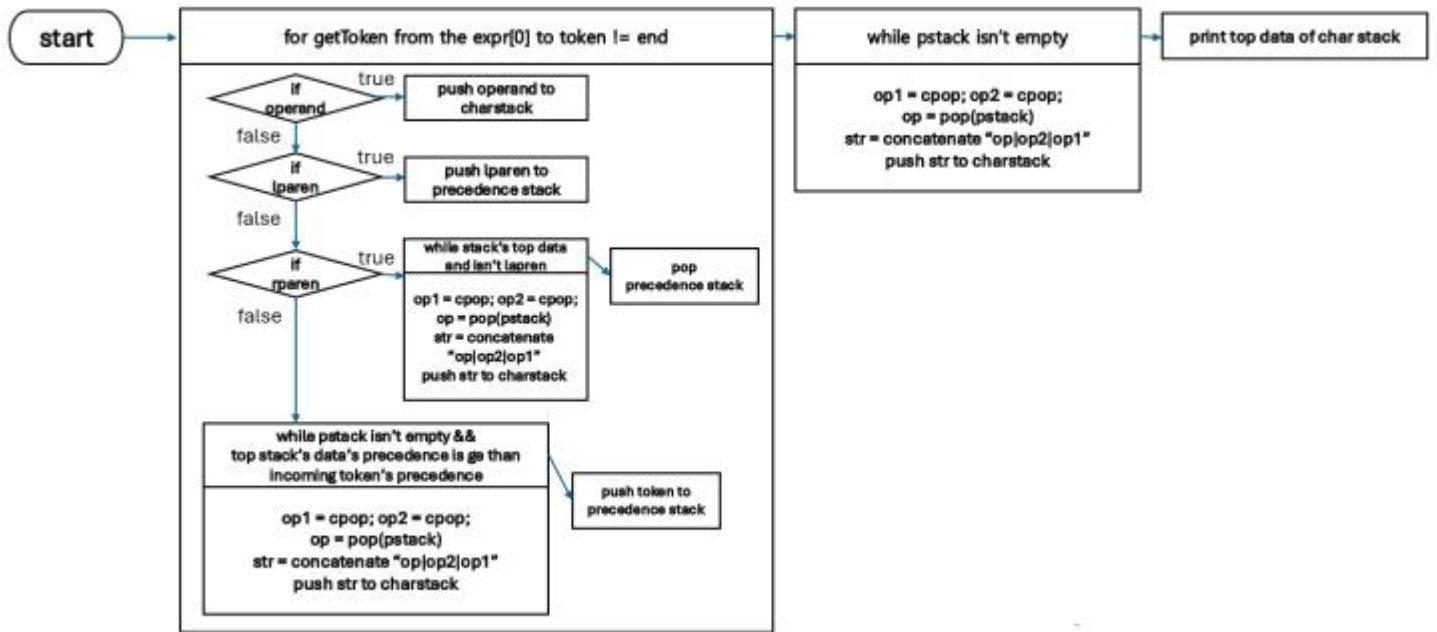
```
precedence getToken(char *symbol, int *n, char* expr)
```

getToken function은 문제1과 동일하게 expr의 n번째 char을 symbol에 저장하고 symbol이 '()+/*% \0-'면 각각에 알맞는 precedence 값을 return 하는 것이다.



```
char printToken(precedence token)
```

printToken function은 precedence자료형인 token을 입력받아 각 token값에 대응하는 char을 return하는 것이다.



```
void prefix(char* expr)
```

prefix function은 infix expression을 인자로 입력받아 prefix expression을 출력하는 것이다.

getToken함수로 expr의 각 char에 대해 token을 구하고(expr의 끝까지)

1. token이 operand면 charstack에 operand를 push한다.

2. token이 lparen이면 lparen을 precedence stack에 push한다.

3. token이 rparen이면 precedence stack이 비어있지 않고 top data가 lparen이 아닌 동안

3-1. op1에 char stack에서 pop한 값을 저장, op2에 char stack에서 pop한 값을 저장, op에 precedence stack에서 pop한 값을 저장한다. 이후 op,op2,op1순서대로 concatenate한 string을 charstack에 push한다. while loop이 끝나면 precedence stack을 pop하여 lparen을 제거한다.

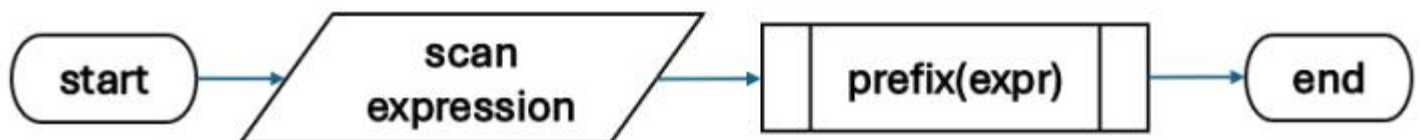
4. token이 위의 3가지 경우가 아니라면 precedence stack이 비어있지 않고 precedence stack top data의 precedence값이 token의 precedence값보다 크거나 같은 동안

4-1. op1에 char stack에서 pop한 값을 저장, op2에 char stack에서 pop한 값을 저장, op에 precedence stack에서 pop한 값을 저장한다. 이후 op,op2,op1순서대로 concatenate한 string을 charstack에 push한다. while loop이 끝나면 token을 precedence stack에 push한다.

expression의 getToken 작업이 끝나면 precedence stack이 비어있지 않는 동안

op1에 char stack에서 pop한 값을 저장, op2에 char stack에서 pop한 값을 저장, op에 precedence stack에서 pop한 값을 저장한다. 이후 op,op2,op1순서대로 concatenate한 string을 charstack에 push한다.

이후 char stack에서 pop한 값을 출력한다. 이 값이 prefix expression이다.



```
int main()
```

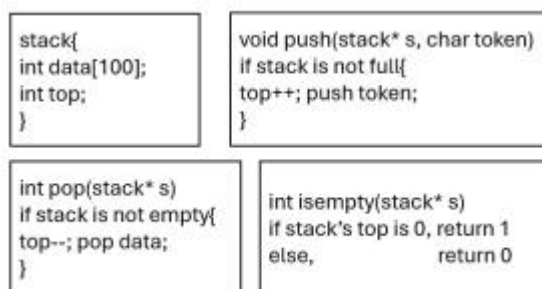
main function은 infix expression을 입력받아 prefix의 인자로 넘겨준다.


```
cse20211562@cspro:~/structure/hw3$ ./a.out
Infix: 3*8+7/1
Prefix: +*38/71
cse20211562@cspro:~/structure/hw3$ ./a.out
Infix: (4-9/5)*(4/1-2)
Prefix: *-4/95-/412
```

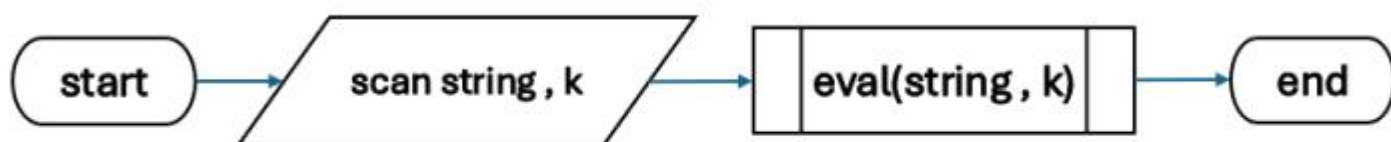
제시된 출력 예시를 만족한다.

문제3

문제 3은 stack을 통해 숫자 num과 이 num에서 삭제할 자리수 k를 입력받아 num에서 k개의 자리수를 제거했을 때 가장 작은 수를 출력하는 문제이다.

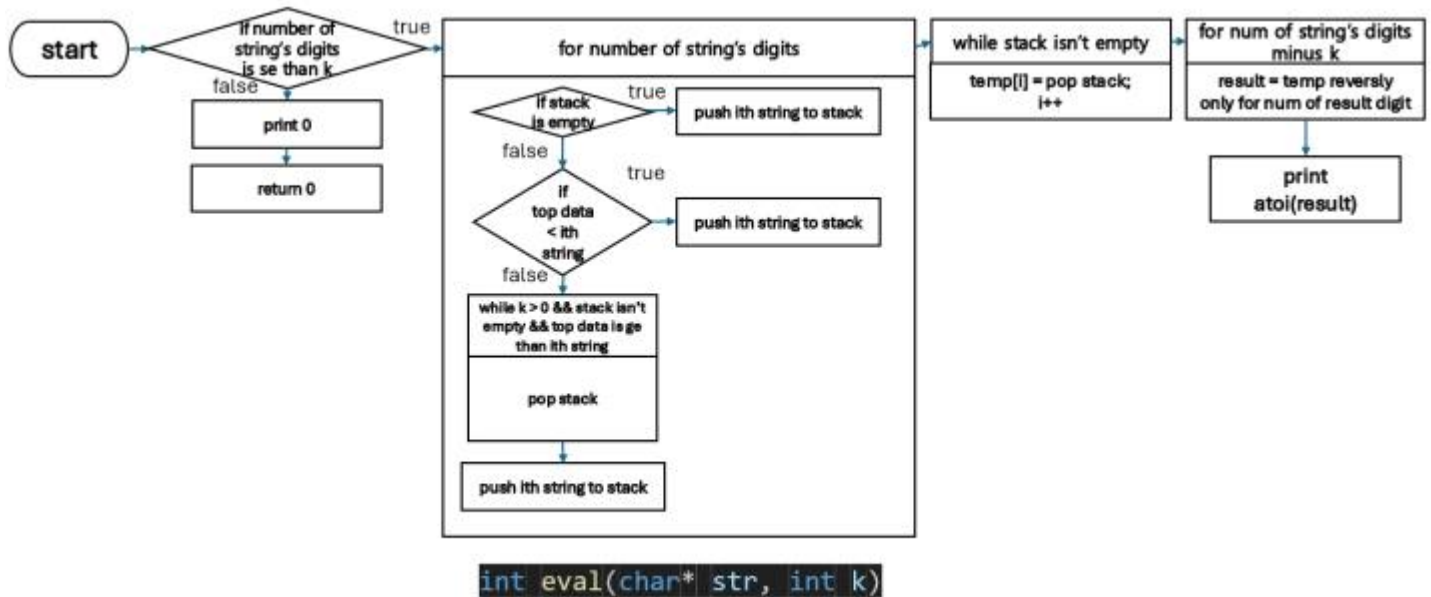


문제3에서 문제를 해결하기 위해 integer 값을 저장하는 stack과 이 stack의 push, pop, isempty function을 구현하였다.



```
int main()
```

main function에서는 자리를 없앨 대상인 숫자를 string으로 입력받아 제거할 자리수인 k와 함께 eval의 인자로 넘긴다.



str의 자리수가 k보다 작거나 같으면 0을 출력한다.(같거나 더 많은 자리수를 삭제하면 0이니) 그렇지 않다면 str의 자리수만큼 for loop을 반복한다.

1. 만약 stack이 비어있다면 str의 i번째 char을 stack에 push한다.
 2. 만약 stack의 top data가 str의 i번째 char보다 작다면 str의 i번째 char을 stack에 push한다.
 3. 위의 두 경우가 아니라면 k가 0보다 크고 stack이 비어있지 않고 stack의 top data가 str의 i번째 char보다 크거나 같은 동안
 - 3-1. stack을 pop한다.(k도 1씩 감소시킨다.) while loop이 끝나면 str의 i번째 char을 stack에 push한다.
- 위의 while loop이 끝나면 temp라는 string에 stack에서 pop한 char를 하나씩 저장한다. 이후 temp에 저장된 digit이 원래 str의 digit에서 k만큼 뺀 값보다 큰 경우를 막기 위해 result라는 string에 temp의 맨 마지막 인덱스부터 우리가 구하는 자리수만큼만 저장한다. 이후 result를 atoi한 값을 출력한다.

```

cse20211562@cspro:~/structure/hw3$ ./a.out
1432219
3
1219
cse20211562@cspro:~/structure/hw3$ ./a.out
10200
1
200
cse20211562@cspro:~/structure/hw3$ ./a.out
10
2
0

```

이는 제시된 출력 예시를 만족한다.